Key Decisions

For Cognition: what I built, why I scoped it the way I did, and where my reasoning changed along the way. Repo: github.com/rythama/internal-tools-platform

The question I actually answered

“Can Devin build these tools?” isn’t really the question — of course it can; CRUD over Postgres was never the hard part. The question worth answering is whether owning ~13 internal tools actually costs less than $250K/year once you account for the platform underneath them, the engineer who has to own that platform, and the compliance surface it drags into scope.

So I scoped the prototype around the eleventh tool instead of the first: one console hosting many tools, where each tool is a spec file sitting on four shared primitives — deny-by-default authorization, hash-chained audit, PII masking with audited break-glass, and maker-checker approvals. I anchored on the KYC queue because it’s the one workflow that forces all four to carry real weight at once. I also deliberately skipped the two things Power Apps is genuinely good at, citizen development and the connector catalog: Devin makes engineer-hours cheap, but it does not turn an ops analyst into an app author, and pretending otherwise seemed like the fastest way to lose the room.

Three decisions I’d defend

Guardrails before agents. I wrote the architecture doc, schema, core contract, and CI by hand before opening a single session, on the theory that an agent’s output is only as good as what the repo can prove about it.

Interfaces first, so sessions could run in parallel. Declaring the core contract up front let two sessions build against it at the same time — one implementing the primitives, one building the console — and when they merged, the console picked up the real implementation with zero edits. That’s the throughput argument demonstrated rather than asserted.

Masking as a structural property. The read API masks internally and takes no unmask parameter, because an API that lets callers opt out of masking is an API that leaks PII eventually.

Where I changed my mind, twice

I went in expecting to recommend building a framework, and the numbers killed it: marginal per-tool cost came out around $40K custom versus $39K done properly in Power Apps, since only about 12% of per-tool work is agent-compressible — requirements, security review, UAT, and cutover don’t care how fast the code was written. The premise my architecture was organized around was wrong, and ARCHITECTURE.md now says so.

Then I moved to “own the primitives, rent the UI,” and a red-team pass found six load-bearing errors in my own model — among them a 3.2x maintenance penalty on Power Apps I’d never justified, and a breakeven computed against a baseline my own first recommendation would have corrected. Fixed properly, the build never breaks even at any tool count, discount rate, or horizon. So the honest recommendation isn’t a savings pitch at all: it’s a capability purchase at roughly $400K/yr, where renting the UI alone buys ~65% of the primitives for $125–145K/yr and owning them buys the last ~30% for another $260–275K/yr. Framed as cost-cutting it dies in the room; framed as what compliance capability is worth, it’s a normal budget conversation.

Time, honestly

The prototype fits the two-hour bound: about 30 minutes of human seeding plus 66 minutes of agent time across six sessions (~7,000 lines, 228 tests, $27 in measured agent cost). The commit history runs longer because analysis, corrections, and presentation polish continued afterward — including a UI design pass that was human-and-assistant work rather than Devin’s, which DEVIN-WORKFLOW.md discloses. I’d rather state that than have anyone reconstruct it from timestamps.

What actually got measured

Two findings, and both cut against me. First, the verification harness proved none of the invariants it advertised: under mutation testing, deleting PII masking left all 54 tests green, and so did defeating the two-person rule and removing the four-eyes gate. The lint rule I’d called “machine-checked” was evadable five different ways, including the exact idiom the repo’s own policy module uses.

Second, review doesn’t scale with lines of code — it scales with load-bearing claims, and reading doesn’t find them. Every confirmed defect was correct-looking code wrong in a single conjunct, and finding them took executing mutations rather than reading diffs. Across six PRs, human review ranged from minutes to essentially none against volume that warranted hours, while CI stayed green the whole way through; the review tax wasn’t so much paid as silently skipped. The fix worked — tests asserting audit rows and state transitions now kill five of five mutants in CI — but only because we measured.

What I’d flag myself on

I over-ranked one finding: the hardcoded session secret is major rather than critical, since the dev role-switcher hands out any role by design. The genuine critical was subtler — a two-person rule counting votes instead of voters. And the prototype demonstrates the primitives, not the recommendation: the crux, binding a rented UI’s queries to a human identity it can’t forge, is proven here only at the API layer, and the database half is the pilot’s week-two gate. That, plus mutation testing, is where I’d spend the next two hours. Not on more tools.